SOFTWARE REQUIREMENTS SPECIFICATION

AI-Powered HRMS
Text-Based Virtual HR Assistant

Requirements Analysis Document

Document control	Details
Document type	Software Requirements Specification (SRS) / Requirements Analysis
Version	1.0 — Consolidated draft for mentor review
Purpose	Define the required system behaviour, scope, constraints, actors, and acceptance criteria
Technology context	Python, FastAPI, PostgreSQL, REST APIs, LLM tool/function calling
LLM provider	Gemini API, as stated in the student capstone requirements; changes require mentor approval

This document defines WHAT the system must do, not a week-by-week implementation schedule.

upcode SOFTWARE LABS ™

Document Purpose and Reading Guide

- Use this document as the baseline for deciding whether a proposed feature belongs in the project.
- Functional requirements define system capabilities; non-functional requirements define quality and constraints.
- Use cases describe expected interactions and alternative/error paths.
- Acceptance criteria define observable evidence that a requirement has been met.
- This is not a sprint plan, coding tutorial, or implementation schedule.

1. Introduction

1.1 Project Description

The AI-Powered HRMS — Text-Based Virtual HR Assistant is a conversational application that enables employees to access selected HR services through natural-language text. The assistant identifies the user's intent, collects required details, invokes predefined tools/functions, calls backend APIs, and presents results returned by the application.

The application uses Python and FastAPI for backend services, PostgreSQL for persistent storage, REST APIs for business operations, and an LLM API for natural-language understanding and tool/function calling. The backend remains responsible for authentication, authorization, validation, business rules, and database transactions.

1.2 Problem Statement

Employees frequently need to check leave balances, submit leave applications, track request statuses, request employment-related documents, and ask about HR procedures. These activities can require repetitive communication with HR and may delay access to current information. A conventional FAQ chatbot cannot reliably retrieve employee-specific records or execute transactions without secure integration with backend services.

The proposed system addresses this need by providing a text-based AI assistant connected to authenticated REST APIs and a PostgreSQL database. It will retrieve authorized employee information, initiate supported HR requests, track their status, and support decisions by authorized human approvers. Responses about employee records and transaction outcomes must be grounded in actual backend results.

1.3 Project Objectives

1. Provide authenticated access to selected HR services.
2. Allow employees to retrieve their own leave balances, leave history, and request statuses.
3. Allow employees to submit leave requests subject to validation and configured business rules.
4. Support authorized approval or rejection of leave requests.
5. Allow employees to request and track employment, salary, experience, and relieving certificates.
6. Support employee HR queries through verified backend data and approved procedural information.
7. Use LLM intent detection and tool/function calling to invoke predefined backend operations.
8. Persist requests, decisions, and statuses in PostgreSQL.
9. Enforce authorization, input validation, error handling, and data-integrity rules.
10. Demonstrate complete end-to-end operation rather than static chatbot responses.

1.4 Intended Users of This Document

Students, project mentors, reviewers, and testers use this SRS to understand project scope, validate design decisions, develop the application against agreed requirements, and assess the final system.

upGrad SOFTWARE LABS ™

2. Overall Description

2.1 Product Perspective

The application combines a text-based AI assistant with conventional backend services. The LLM is a language and orchestration component, not the authoritative source of employee data and not an authorization or approval authority.

2.2 Product Functions

- Authenticate users and identify their roles.
- Retrieve permitted employee information and leave balances.
- Create and track leave requests.
- Allow authorized approvers to decide eligible pending leave requests.
- Create, process, and track HR document requests.
- Handle HR queries by invoking appropriate backend tools.
- Return clear confirmations, clarification questions, validation messages, and safe error responses.

2.3 User Classes

User class	Description
Employee	Uses the assistant to retrieve permitted personal HR information, submit requests, and track their own requests.
HR / Authorized Approver	Reviews permitted employee requests, makes authorized leave decisions, and processes HR document requests.
Administrator	Manages user accounts, roles, and approved system configuration where those capabilities are included.
AI Assistant	Interprets text and selects approved tools; it does not have independent authority to bypass backend controls.

2.4 Operating Environment and Technology Constraints

Component	Requirement / baseline
Programming language	Python
Backend framework	FastAPI
Database	PostgreSQL is the consolidated baseline from the detailed student documents; an alternative requires mentor approval.
LLM integration	Gemini API and its supported function-calling capability, as specified in the student capstone brief.
Data validation	Typed request/response schemas, such as Pydantic.
Database access	ORM such as SQLAlchemy is proposed; mentor may approve an equivalent.
Authentication	JWT or another explicitly approved secure mechanism; the supplied student documents commonly specify JWT.
API verification	Swagger UI and/or Postman.
Interface	A graphical frontend is optional; a working text-based assistant and backend workflows are mandatory.

2.5 Assumptions and Dependencies

- Students have already studied Python, FastAPI, SQL/database concepts, REST APIs, and API integration.
- The project will use synthetic employee and HR data rather than real employer personal data.
- LLM API access and credentials will be available to the team.
- Exact leave policies, approval hierarchy, and document-processing rules must be confirmed before implementation.
- Optional RAG policy search is included only if the mentor confirms it is part of the required scope.

Uncode SOFTWARE LABS ™

3. Scope Definition

3.1 In Scope

- Authentication and protected API access.
- Role-based authorization and employee record ownership checks.
- Employee profile lookup for permitted fields.
- Leave balance retrieval.
- Leave request creation, validation, history/status retrieval, and authorized approval/rejection.
- HR certificate request creation and status tracking for four supported document types.
- Employee HR queries and pending/previous request tracking.
- Natural-language intent detection, clarification, and allow-listed tool/function calling.
- Database persistence, input validation, consistent errors, and audit-relevant records.
- API documentation, tests, setup guidance, and end-to-end acceptance evidence.

3.2 Out of Scope for Initial Release

- Payroll processing or salary disbursement.
- Attendance devices or biometric integration.
- Recruitment and applicant tracking.
- Performance appraisal.
- Automatic approval of HR requests by the LLM.
- Automatic certificate generation, digital signing, or delivery unless separately approved.
- Email, SMS, or WhatsApp notifications unless separately approved.
- Production integration with an existing employer HR database.
- Voice interaction or a mandatory full-featured web/mobile frontend.

3.3 Feature — HR Policy RAG

One supplied detailed SRS includes Retrieval-Augmented Generation (RAG) over approved HR policy documents, with source references and version metadata. The short original capstone brief does not explicitly require RAG. Therefore, RAG is classified as optional pending mentor confirmation. If approved, its requirements should be added to the agreed scope before implementation.

4. Actors and Authorization Requirements

Capability	Employee	HR / Approver	Administrator
Sign in and view permitted own profile	Yes	Yes	Yes
View own leave balance and own requests	Yes	Yes	Yes, subject to policy
Submit own leave request	Yes	If permitted	If explicitly enabled
Approve/reject leave	No, unless explicitly assigned as approver and not self-approving	Yes, if assigned permission	Only if explicitly granted
Create own HR document request	Yes	If permitted	If explicitly enabled
Process HR document requests	No	Yes, if authorized	If assigned permission
Manage users and roles	No	No by default	Yes
View audit logs	No	Only if explicitly authorized	Yes, subject to audit policy

Authorization must be enforced by backend services on every protected endpoint and tool invocation. Role names alone do not replace record-ownership checks.

4.1 Identity and Access Requirements

- An employee ID supplied in a chat message or request body must not be treated as proof of identity.

- The authenticated identity must be derived from a verified token/session.
- Employees must not access another employee's private records by changing a request identifier.
- Approval and document-processing actions must require explicit backend permissions.
- The AI model must not be allowed to set its own permissions or bypass application authorization.

5. Functional Requirements

ID	Requirement area	Requirement statement	Priority
FR-01	Authentication	The system shall authenticate registered employees and authorized HR/admin users before allowing protected operations.	High
FR-02	Token/session validation	The backend shall reject missing, invalid, or expired credentials on protected APIs.	High
FR-03	Role-based authorization	The backend shall enforce permissions for each protected API and AI tool invocation.	High
FR-04	Employee data isolation	Employees shall retrieve or modify only their own permitted records unless a specific permission allows otherwise.	High
FR-05	Employee profile retrieval	The system shall retrieve permitted profile information for the authenticated employee.	High
FR-06	Leave balance retrieval	The system shall return leave entitlement, used days, and remaining balance by leave type/year where those fields are supported.	High
FR-07	Leave detail collection	The assistant shall identify leave type, start date, end date, and other mandatory details; it shall ask for missing or ambiguous information.	High
FR-08	Leave validation	The backend shall validate date ranges, active employee status, leave eligibility, available balance, and approved leave-type rules before request creation.	High
FR-09	Leave request creation	The system shall store a valid leave request and return a unique tracking identifier and initial status.	High
FR-10	Leave history and status	The system shall allow an employee to retrieve their own previous requests and the current status of a specific request.	High
FR-11	Leave approval/rejection	Only authorized approvers shall approve or reject eligible pending requests; decision, actor, time, and optional remarks shall be recorded.	High
FR-12	Leave balance consistency	Leave usage/balance shall be updated according to the approved policy only after approval; rejection shall not consume approved leave balance.	High
FR-13	Duplicate decision protection	The system shall prevent duplicate decisions and duplicate leave deductions for the same request.	High

FR-14	Document type support	The system shall support Employment, Salary, Experience, and Relieving Certificate requests.	High
FR-15	Document request creation	The system shall validate and persist a supported HR document request and return its tracking identifier and status.	High
FR-16	Document request processing	Authorized HR personnel shall update document request status and permitted remarks, with processor and timestamp recorded.	High
FR-17	Document status retrieval	Employees shall retrieve the status of their own document requests; authorized HR may retrieve requests allowed by role.	High
FR-18	HR query handling	The assistant shall support questions about leave balances, previous applications, request statuses, pending HR requests, and supported HR procedures.	High
FR-19	Intent detection	The assistant shall identify the intended supported HR operation from natural-language text.	High
FR-20	Tool/function calling	The assistant shall invoke only predefined, allow-listed tools with schema-validated arguments.	High
FR-21	Backend-grounded responses	Employee-specific responses and transaction confirmations shall be based on actual backend results; the assistant shall not invent records or outcomes.	High
FR-22	Input and business-rule validation	The backend shall validate required fields, data types, supported values, dates, and allowed status transitions.	High
FR-23	Error handling	The system shall return consistent, safe responses for validation errors, unauthorized actions, missing records, conflicts, and service failures.	High
FR-24	Persistence	Successfully created requests, decisions, and status changes shall remain stored and be retrievable later.	High
FR-25	Audit logging	The system shall record security-relevant events and important state changes without logging passwords, tokens, or secrets.	High
FR-26	Secret management	LLM credentials, database credentials, and signing keys shall be kept out of source control and user-facing responses.	High
FR-27	API documentation	The application shall expose API documentation suitable for development and testing.	Medium
FR-28	Health check	The application shall provide a health-check endpoint for verifying service availability.	Medium
FR-29	Optional policy RAG	If approved for scope, the system shall answer policy questions using approved documents, source metadata, and an uncertainty fallback when evidence is insufficient.	Optional / confirm

upcode SOFTWARE LABS ™

6. Business Rules

ID	Rule
BR-01	Only authenticated and active users may access protected HR functions.
BR-02	Employees may view only their own private HR records unless explicitly authorized otherwise.
BR-03	Leave requests must include valid dates and all mandatory fields.
BR-04	Leave eligibility and balance must be validated by backend business logic, not decided by the LLM.
BR-05	Leave balance is deducted only after authorized approval, according to the confirmed leave policy.
BR-06	Approval status and balance updates must remain consistent; duplicate deductions are not allowed.
BR-07	Only eligible pending requests may be approved or rejected.
BR-08	Every successfully created leave/document request must receive a unique tracking identifier.
BR-09	The assistant must not claim success unless the backend confirms the operation.
BR-10	Unsupported document types must be rejected with a clear explanation.
BR-11	Policy answers using RAG, if included, must be grounded in approved sources and acknowledge insufficient evidence.
BR-12	Secrets and plaintext passwords must not be exposed in prompts, logs, API responses, or source control.

6.1 Business Decisions Requiring Approval

Decision	Question / proposed baseline
Casual leave notice	One detailed SRS proposes at least two calendar days' notice. Confirm whether this is the intended rule and make it configurable.
Leave types	Confirm casual, annual, sick, and any other supported types.
Leave calculation	Confirm calendar days versus working days, holidays/weekends, partial days, and yearly entitlement.
Approval hierarchy	Confirm who can approve, whether managers and HR both have authority, and whether self-approval is prohibited.
Balance calculation	Confirm when balance is reserved/deducted and how concurrent pending requests are handled.
Document workflow	Confirm permitted statuses and whether the scope is tracking only or includes processing/completion.
Authentication details	Confirm JWT/session expiry, logout/revocation behaviour, and account registration policy.
LLM provider	Gemini is the student capstone baseline; any change requires mentor approval.
RAG	policy document retrieval is required

7. Use Cases

UC-01 — Sign In

Primary actor: Employee or HR/Admin. Preconditions: account exists and is active. Main flow: user submits credentials; backend verifies them; a valid token/session is returned. Alternative: invalid credentials or inactive account is rejected. Acceptance: protected endpoints cannot be accessed without valid authentication.

UC-02 — Check Leave Balance

Primary actor: Employee. Preconditions: authenticated employee and balance record exist. Main flow: user asks for leave balance; assistant identifies intent; backend derives identity from authentication, retrieves the relevant balance, and assistant displays the returned values. Alternative: assistant asks for leave type/year if unclear. Acceptance: result matches stored data and no other employee's balance is exposed.

UC-03 — Submit Leave Request

Primary actor: Employee. Preconditions: authenticated, active account. Main flow: assistant gathers leave type/dates/required details; backend validates policy, dates, and balance; valid request is saved with a tracking ID and Pending status. Alternatives: missing fields trigger clarification; invalid dates or insufficient eligibility prevent creation. Acceptance: invalid request creates no record and pending is not described as approved.

UC-04 — Approve or Reject Leave

Primary actor: Authorized approver. Preconditions: authenticated approver has permission and request is eligible/pending. Main flow: approver selects decision; backend checks permission and state; decision and related balance changes are recorded consistently. Alternatives: unauthorized or repeated decision is denied safely. Acceptance: decision is auditable and balance is not deducted twice.

UC-05 — Request HR Document

Primary actor: Employee. Preconditions: authenticated employee. Main flow: user requests a supported certificate; assistant identifies type and gathers required details; backend validates and stores request; assistant returns ID/status. Alternative: unsupported type is rejected. Acceptance: request is persistent and trackable.

UC-06 — Track Request

Primary actor: Employee or authorized HR. Main flow: user asks for a request status; backend retrieves the record after ownership/permission checks; assistant displays the current stored status. Acceptance: access to another employee's request is blocked.

UC-07 — View Previous/Pending HR Requests

Primary actor: Employee. Main flow: assistant identifies history/pending query; backend retrieves only the authenticated employee's permitted requests; assistant summarizes actual results. Acceptance: results match database records.

UC-08 — Ask HR Procedure Question

Primary actor: Employee. Main flow: assistant answers using approved implemented procedural information. If RAG is approved, it retrieves approved policy passages and cites source metadata. Alternative: if evidence is insufficient, it states that a reliable answer is unavailable and recommends contacting HR.

UC-09 — Process Document Request

Primary actor: Authorized HR. Main flow: HR retrieves a permitted document request, changes status/remarks through an authorized API, and backend records processor/time. Acceptance: employee can later retrieve the updated status.

8. External Interface and API Requirements

The following are proposed endpoint examples to make the required capabilities testable. They are not claims that these endpoints already exist, and students may use different names if the approved API contract provides equivalent behaviour.

Proposed method and endpoint	Purpose	Access rule
POST /auth/login	Authenticate and issue token/session	Login policy
POST /auth/logout	Invalidate session/token where supported	Authenticated
GET /employees/me	Retrieve current user's permitted profile	Authenticated
GET /leave-balances/me	Retrieve own leave balance	Employee's own data
GET /leave-requests/me	List own leave requests	Employee's own data
POST /leave-requests	Create leave request	Authenticated employee

GET /leave-requests/{id}	Retrieve request and status	Owner or authorized HR
POST /leave-requests/{id}/decision	Approve/reject eligible pending request	Authorized approver
POST /document-requests	Create HR document request	Authenticated employee
GET /document-requests/me	List own document requests	Employee's own data
GET /document-requests/{id}	Retrieve document status	Owner or authorized HR
PATCH /document-requests/{id}	Process document request status/remarks	Authorized HR
POST /chat	Submit text to assistant	Authenticated when protected HR data/actions are involved
GET /health	Service health	As defined by deployment policy

8.1 API Response Requirements

- Request/response bodies must be documented and validated.
- Use appropriate status codes, including 200/201 for success, 401 for unauthenticated, 403 for forbidden, 404 for missing resources, 409 for conflicting state, and 422 for validation failures as appropriate.
- Error responses must be safe and must not expose secrets or internal stack traces.
- Identity and role must be derived from verified authentication context, not trusted from model-generated arguments.

9. Data Requirements

The following entities are suggested by the supplied documents. The final schema and field definitions require an ER diagram and mentor review.

Entity	Purpose / indicative data
Employees	Employee identifier/code, permitted profile data, active status; authentication credentials may be kept in a separate users table.
Users / Roles (recommended)	Login identity, password hash or identity reference, active status, roles/permissions.
Leave Balances	Employee, leave type, leave year, entitlement, used days, timestamps.
Leave Requests	Requester, leave type, dates, days requested, reason where required, status, tracking ID, timestamps.
Leave Approvals	Leave request, approver, decision, remarks, decision timestamp.
HR Document Requests	Requester, supported document type, purpose, status, remarks, processor, timestamps.
Audit Logs (recommended)	Actor, action, target record, outcome, timestamp and safe metadata.
Conversation history (optional)	User ownership and retention metadata, only if conversation storage is required.
Policy documents/chunks (optional RAG)	Approved document title/version/effective date/source and indexed text/embedding.

- Use primary/foreign keys, uniqueness and appropriate constraints.
- Do not store plaintext passwords.
- Use transactions for related status and balance changes.
- Define retention and access rules for personal data, conversations, and logs.

10. Non-Functional Requirements

ID	Requirement
NFR-01 Security	Use secure password hashing, token/session validation, least privilege, backend authorization, and safe secret storage.
NFR-02 Privacy	Restrict employee records by ownership/role; use synthetic data; define retention for messages, logs, and documents.
NFR-03 Reliability	Related approval and balance operations must be transactional and recover safely from failures.
NFR-04 Performance	Set measurable performance targets during testing. One supplied SRS proposes common non-LLM local API calls typically respond within one second; confirm this as a target rather than a guarantee.
NFR-05 AI availability	Handle LLM provider timeouts/failures with a safe fallback; keep

	ordinary backend APIs usable where possible.
NFR-06 Maintainability	Separate routes, schemas, models, services, configuration, and AI integration; use automated tests.
NFR-07 Observability	Use safe structured logs, request/correlation IDs, health checks, and audit events where appropriate.
NFR-08 Usability	Provide readable confirmations and validation errors; ask for clarification rather than guessing.
NFR-09 Portability	Document setup instructions and avoid hard-coded machine-specific paths.
NFR-10 Data integrity	Use relationships, uniqueness/check constraints, transactions, and a documented database setup/restore approach.
NFR-11 Documentation	Provide requirements, API documentation, setup instructions, and evidence sufficient for another student/mentor to run and verify the application.

11. Error and Exception Requirements

Condition	Expected behaviour
Missing/invalid/expired token	Return an authentication error; do not return protected data.
Insufficient permission	Deny the operation; return a safe forbidden/not-found response according to the chosen privacy policy.
Missing required field	Ask a clarification question in chat or return a clear validation error through the API.
Invalid dates / policy violation	Reject the request with a field-specific explanation; do not create an invalid record.
Insufficient leave balance	Do not create an eligible request unless the approved policy explicitly supports an exception path.
Request not found	Return a safe not-found response.
Already-decided request	Reject a repeated decision as a conflict and preserve the original decision.
LLM unavailable	Return a safe fallback; do not fabricate a result.
Database failure	Roll back the affected transaction, return a safe error, and log technical details without exposing secrets.
No approved policy evidence (if RAG enabled)	State that approved sources do not provide a reliable answer; do not invent a policy.

12. Verification and Acceptance Criteria

The requirements are accepted when the following observable conditions can be demonstrated. These criteria define expected behaviour; they do not prescribe a development schedule.

☐ Protected endpoints reject unauthenticated requests.

☐ Employees cannot access another employee's private records by changing IDs or tool arguments.

☐ An employee can retrieve the actual leave balance associated with their authenticated account.

☐ A valid leave request is persisted with a unique tracking ID and an initial status defined by the approved workflow.

☐ Invalid leave requests are rejected without creating invalid records.

☐ Request status can be retrieved after the original creation interaction has ended.

☐ Only authorized approvers can decide eligible pending requests.

☐ Approval updates leave usage consistently; rejection does not deduct approved leave; repeat decisions do not duplicate deductions.

☐ Each supported certificate type can be requested and tracked.

☐ Authorized HR can process document requests according to the approved status model.

☐ The assistant uses actual API results and does not invent IDs, balances, statuses, or successful operations.

☐ The assistant asks for missing required details and handles unsupported requests clearly.

☐ Errors are safe and consistent; secrets and plaintext passwords are not exposed.

☐ API docs, setup instructions, test evidence, and the agreed technical deliverables are available.

☐ If RAG is approved, policy responses use approved sources, cite available source metadata, and acknowledge insufficient evidence.

13. Requirements Traceability

Each requirement should be traceable to implementation and test evidence. Students may extend this table during development.

Requirement IDs	Verification evidence
FR-01 to FR-04	Authentication and authorization tests; cross-employee access tests.
FR-05 to FR-10	API tests and database verification for profile, balance, leave creation, history and status.
FR-11 to FR-13	Approval/rejection tests, permission tests, transaction and duplicate-deduction tests.
FR-14 to FR-17	Document creation, processing, ownership and status-retrieval tests.
FR-18 to FR-21	AI/tool-calling tests using natural-language queries and actual backend responses.
FR-22 to FR-26	Validation, safe-error, persistence, audit and secret-handling checks.
FR-27 to FR-28	Swagger/OpenAPI availability and health-check test.
FR-29	Only applicable if policy RAG is approved; retrieval, citation and insufficient-evidence tests.

upcode™ SOFTWARE LABS